

Complete Textbook Notes on Day9 to do

Google Career Preparation

A professional textbook-style edition of the complete Day 9 planning conversation

Coverage

Day 9 objectives • Trees • DFS/BFS • DSA problems • Backend query abstraction • API reliability • System design • Documentation • Git discipline • Definition of Done

Prepared for long-term study, revision, teaching, and interview reference

Verbatim conversation content is preserved in sequence; textbook styling, navigation, and supplementary study aids are added around it.

Table of Contents

The original Day 9 plan is reproduced in its original sequence in Part I. The chapter structure below is added for textbook navigation.

1. 1. DAY 9 OBJECTIVES	2	23. 23. BLOCK 5 — PROJECT 1 IMPLEMENTATION	2
2. 2. WHAT DAY 9 IS NOT	2	24. 24. Task 2 — Introduce a query representation	2
3. 3. TIME-BLOCKED DAY 9	2	25. 25. Task 3 — Refactor repository query handling	2
4. 4. BLOCK 1 — REPOSITORY BASELINE	2	26. 26. Task 4 — Preserve API behavior	2
5. 5. BLOCK 2 — TREE FUNDAMENTALS	2	27. 27. Task 5 — Test regression	2
6. 6. BINARY TREE	2	28. 28. BLOCK 6 — API RELIABILITY	2
7. 7. DFS	2	29. 29. DATABASE UNAVAILABLE	2
8. 8. TREE DFS TRAVERSALS	2	30. 30. TIMEOUTS	2
9. 9. BFS	2	31. 31. IDEMPOTENCY — CONCEPTUAL	2
10. 10. DFS VS BFS	2	32. 32. RELIABILITY TESTS	2
11. 11. TREE GRAPH CONNECTION	2	33. 33. BLOCK 7 — SYSTEM DESIGN	2
12. 12. BLOCK 3 — DSA PROBLEMS	2	34. 34. SYSTEM DESIGN REQUIREMENTS	2
13. 13. DSA PROBLEM 1	2	35. 35. CONNECT IT TO PROJECT 1	2
14. 14. DSA PROBLEM 2	2	36. 36. BLOCK 8 — INTERVIEW DRILL	2
15. 15. DSA PROBLEM 3	2	37. 37. GOOGLE-STYLE PROJECT INTERVIEW	2
16. 16. REQUIRED DSA FORMAT	2	38. 38. GOOGLE-INTERVIEW-PREP REPOSITORY	2
17. 17. TREE PATTERN-RECOGNITION DRILL	2	39. 39. PROJECT 1 DOCUMENTATION	2
18. 18. BLOCK 4 — BACKEND	2	40. 40. GIT DISCIPLINE	2
19. 19. THE PROBLEM	2	41. 41. DAY 9 DEFINITION OF DONE	2
20. 20. DAY 9 QUERY OBJECT	2	42. 42. WHAT NOT TO DO ON DAY 9	2
21. 21. IMPORTANT BOUNDARY	2	43. 43. THE MOST IMPORTANT DAY 9 ENGINEERING LESSON	2
22. 22. SAFE QUERY CONSTRUCTION	2		

Part I — Verbatim Day 9 Planning Conversation

Original user request

I have completed all the tasks from Day 1 till Day8.

Refer the 12 month blueprint and give me the to do for the Day9. Do not repeat tasks.

Yes. Since you have explicitly completed Days 1–8, Day 9 should move forward rather than revisit completed work.

I used the 12-month blueprint together with the completed Day 8 plan. Day 8 ended with the DSA progression moving toward Trees BFS/DFS, while Project 1 had reached PostgreSQL + CRUD + filtering + pagination + ordering + query abstraction. fileciteturn0file7L31-L33

The 12-month blueprint also says Month 1 should establish Python/SWE foundations, Month 2 DSA, and Month 3 should combine DSA + FastAPI/PostgreSQL/backend + Project 1 Document Service. Therefore, Day 9 should remain firmly in the SWE/backend + DSA depth phase rather than jumping prematurely into Redis, Docker, Kubernetes, GCP, ML, RAG, or agents.

GOOGLE CAREER PREPARATION — DAY 9

Core Theme

Trees + BFS/DFS + Backend Query Abstraction + API Reliability

Target: ~3 hours

Day 9 progression

```

text
Day 1
Python + SWE foundations
  ↓
Day 2
FastAPI + layered backend
  ↓
Day 3
Revision
  ↓
Day 4
Backend architecture + Sliding Window
  ↓
Day 5
PostgreSQL persistence
  ↓
Day 6
PostgreSQL verification + Prefix Sum/Hashing
  ↓
Day 7
Hashing + Intervals
  ↓

```

Day 8
Stacks + Queues + SQL querying
↓
Day 9
Trees + BFS/DFS
+
Backend query abstraction
+
API reliability

Day 8 specifically established the DSA progression:

Two Pointers Sliding Window Prefix Sum Hashing Intervals Stack Monotonic Stack Queue
Trees/BFS/DFS. fileciteturn0file7L31-L33

1. DAY 9 OBJECTIVES

By the end of Day 9, you should be able to:

DSA

- ☐ Understand what a tree is.
- ☐ Understand binary trees.
- ☐ Understand root, parent, child, leaf, depth and height.
- ☐ Understand recursive DFS.
- ☐ Understand iterative DFS.
- ☐ Understand BFS/level-order traversal.
- ☐ Recognize when DFS applies.
- ☐ Recognize when BFS applies.
- ☐ Understand the relationship between a tree and a graph.
- ☐ Understand recursion stack usage.
- ☐ Solve 3 new tree problems.
- ☐ Explain each solution without looking at code.
- ☐ Record complexity and invariants.

Backend

Day 8 already implemented query parameters, pagination, filtering, ordering and parameterized SQL.
fileciteturn0file7L23-L30

Therefore do not implement those again.

Day 9 moves one level deeper:

- ☐ Separate query parameters from query-building logic.
- ☐ Understand repository query composition.
- ☐ Understand typed query/filter objects.
- ☐ Understand safe dynamic SQL construction.
- ☐ Understand deterministic ordering.

- ☐ Understand repository method boundaries.
- ☐ Understand how to avoid an increasingly large repository method.
- ☐ Introduce a clean query abstraction only if the current code genuinely needs it.

Reliability

- ☐ Understand request-level error handling.
- ☐ Understand database failure propagation.
- ☐ Understand timeout concepts.
- ☐ Understand idempotency conceptually.
- ☐ Understand API behavior during partial failures.
- ☐ Add targeted reliability tests.
- ☐ Do not build distributed infrastructure yet.

System Design

- ☐ Design a tree/document hierarchy API.
- ☐ Connect BFS/DFS concepts to backend data traversal.
- ☐ Explain how the current Project 1 could evolve into hierarchical engineering artifacts.

2. WHAT DAY 9 IS NOT

This is important because your roadmap is deliberately staged.

Do not repeat:

- Two Pointers
- Sliding Window
- Prefix Sum
- Intervals
- Stack
- Queue
- PostgreSQL installation
- PostgreSQL CRUD
- PostgreSQL persistence verification
- basic filtering
- basic pagination
- basic ordering
- basic parameterized SQL

Those are already Day 1–8 material.

Day 8 already explicitly established pagination, filtering, ordering, parameterized SQL, repository query responsibilities and testing. fileciteturn0file7L17-L20

3. TIME-BLOCKED DAY 9

Block	Time	Activity
1	10 min	Repository baseline
2	30 min	Tree fundamentals
3	50 min	3 Tree/BFS/DFS problems
4	25 min	Backend query abstraction
5	40 min	Project 1 implementation
6	15 min	Reliability tests
7	10 min	System design
8	10 min	Interview + documentation
Total	190 min	~3h 10m

If you only have exactly 3 hours, reduce documentation/interview time.

4. BLOCK 1 — REPOSITORY BASELINE

Time: 10 minutes

Do this before modifying Project 1.

Run:

```
bash
git status
git log --oneline -n 10
python -m pytest -q
```

Then inspect:

```
text
src/
tests/
docs/
```

Specifically locate:

```
text
document_routes
document_service
document_repository
database/connection
Pydantic models
```

Goal

Understand what your actual Day 8 implementation contains.

Do not assume the Day 8 plan and your code are identical.

5. BLOCK 2 — TREE FUNDAMENTALS

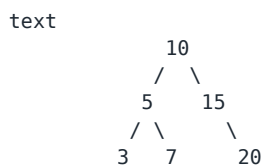
Time: 30 minutes

This is the major new DSA concept.

5.1 What is a Tree?

A tree is a hierarchical data structure.

Example:

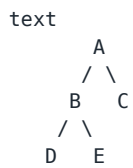


Understand:

- root
 - parent
 - child
 - sibling
 - leaf
 - subtree
 - depth
 - height
-

6. BINARY TREE

A binary tree allows each node to have at most two children:



Each node has:

```

text
value
left
right
  
```

Conceptually:

```

python
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
  
```

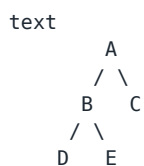
Do not turn this into a large OOP exercise.

The goal is understanding the data structure.

7. DFS

Depth-First Search explores one branch deeply before moving to another.

Conceptually:



A DFS can produce:

```

text
A
B
D
E
C
  
```

You need to understand that DFS is naturally expressed through recursion.

8. TREE DFS TRAVERSALS

Understand the three fundamental traversals.

Preorder

```
text
Root
Left
Right
```

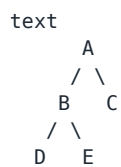
Inorder

```
text
Left
Root
Right
```

Postorder

```
text
Left
Right
Root
```

For:



Preorder:

```
text
A B D E C
```

Inorder:

```
text
D B E A C
```

Postorder:

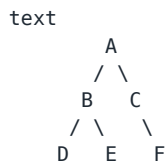
```
text
D E B C A
```

You should know why the order changes rather than memorizing sequences.

9. BFS

Breadth-First Search explores the tree level by level.

Example:



BFS:

```

text
A
B C
D E F
  
```

Use:

```

text
Queue
  
```

This directly connects to the queue concept you completed on Day 8.

Day 8 established that queues are used for level-order processing and BFS. fileciteturn0file7L6-L7

10. DFS VS BFS

You should be able to explain:

Property	DFS	BFS
Primary structure	Recursion/stack	Queue
Exploration	Deep first	Level first
Tree traversal	Yes	Yes
Typical use	Path/subtree exploration	Level/shortest unweighted path
Space	Depends on depth	Depends on breadth

Do not memorize this as a table.

Understand the underlying traversal behavior.

11. TREE GRAPH CONNECTION

This is an important Google interview concept.

A tree is a special type of graph:

```
text
Graph
  ↓
Connected + acyclic
  ↓
Tree
```

For Day 9, understand this conceptually.

You do not need to start general graph algorithms yet.

The progression remains:

```
text
Trees
  ↓
BFS / DFS
  ↓
Graphs
  ↓
Topological Sort
  ↓
Union-Find
```

12. BLOCK 3 — DSA PROBLEMS

Time: 50 minutes

Use three new problems.

Do not reuse Day 8 problems.

13. DSA PROBLEM 1

Binary Tree Preorder Traversal

Pattern

text
Tree DFS

Objective

Return the preorder traversal of a binary tree.

Example:

```

text
  1
   \
    2
   /
  3

```

Output:

```

text
[1, 2, 3]

```

Learn

- recursive DFS
- base case
- visit root
- traverse left
- traverse right
- recursion stack

Then implement an iterative version using an explicit stack.

Interview objective

Be able to answer:

Why does the recursive implementation use $O(h)$ auxiliary space?

where h is tree height.

14. DSA PROBLEM 2

Binary Tree Level Order Traversal

Pattern

text
BFS + Queue

Example:

text

```

      3
     / \
    9  20
   /  \
  15   7

```

Output:

text

```

[
  [3],
  [9, 20],
  [15, 7]
]

```

Learn

The critical concept is:

Process one complete level before moving to the next level.

Use:

text
queue

At each iteration:

1. determine current level size
 2. process exactly that many nodes
 3. add children
 4. construct the next level
-

15. DSA PROBLEM 3

Maximum Depth of Binary Tree

Pattern

text
DFS / recursion

For:

text

```

      3
     / \
    9  20
   /  \
  15   7
  
```

the maximum depth is:

text
3

Key recurrence

Conceptually:

text

```

depth(node)
=
1 + max(
    depth(left),
    depth(right)
)
  
```

Base case:

text
None → 0

Interview objective

Explain why every node is processed once.

Target:

text
Time: $O(n)$
Space: $O(h)$

for recursive DFS.

16. REQUIRED DSA FORMAT

For each of the three problems:

```
text
Problem
↓
Pattern
↓
Recognition signal
↓
Brute force / naive reasoning
↓
Why it is insufficient
↓
Optimized approach
↓
Invariant / recurrence
↓
Algorithm
↓
Dry run
↓
Complexity
↓
Edge cases
↓
Common mistakes
↓
Alternative approach
↓
Interview follow-ups
```

This continues your Google interview preparation methodology.

17. TREE PATTERN-RECOGNITION DRILL

After solving the problems, do this without code.

Scenario A

"Visit every node as deeply as possible before exploring siblings."

Answer:

```
text
DFS
```

Scenario B

"Return nodes grouped by level."

Answer:

```
text
BFS
```

Scenario C

"Find the maximum depth."

Answer:

text
DFS or BFS

Scenario D

"Process all nodes at distance 1, then distance 2, then distance 3."

Answer:

text
BFS

Scenario E

"Explore a subtree recursively."

Answer:

text
DFS

Scenario F

"What data structure naturally supports level-order traversal?"

Answer:

text
Queue

18. BLOCK 4 — BACKEND

QUERY ABSTRACTION

Time: 25 minutes

This is the Project 1 continuation from Day 8.

Day 8 established:

```
text
Router
  ↓
Service
  ↓
Repository
  ↓
PostgreSQL
```

and specifically required that SQL remain in the repository rather than the router. fileciteturn0file7L18-L20

Day 9 should not immediately add more endpoints.

Instead, examine whether your repository is beginning to accumulate logic such as:

```
text
get_all()
get_filtered()
get_sorted()
get_paginated()
get_filtered_sorted()
get_filtered_paginated()
...
```

That can become difficult to maintain.

19. THE PROBLEM

Imagine your repository eventually has:

```
python
get_all_documents()

get_documents_by_type()

get_documents_paginated()

get_documents_by_type_paginated()

get_documents_sorted()

get_documents_by_type_sorted()

get_documents_by_type_sorted_paginated()
```

This creates combinatorial growth.

You need to understand a better abstraction.

20. DAY 9 QUERY OBJECT

Introduce the concept of a query specification.

Conceptually:

```
text
DocumentQuery
-----
document_type
status
limit
offset
sort_field
sort_order
```

Then:

```
text
Router
  ↓
Query parameters
  ↓
Service
  ↓
DocumentQuery
  ↓
Repository
  ↓
SQL
```

The key principle:

Represent the query as structured data rather than creating a separate repository method for every combination.

21. IMPORTANT BOUNDARY

The router should still own HTTP concerns:

```
text
HTTP query parameters
validation
HTTP response
```

The service should own:

```
text
business validation
orchestration
```

The repository should own:

```
text
SQL
query construction
```

database execution
record mapping

This preserves the architecture established on Day 8. fileciteturn0file7L17-L19

22. SAFE QUERY CONSTRUCTION

Continue the security principle from Day 8.

Values can be parameterized:

```
sql
WHERE document_type = %s
```

But SQL identifiers such as:

```
text
created_at
name
```

must come from an allowlist.

Conceptually:

```
python
ALLOWED_SORT_FIELDS = {
    "created_at": "created_at",
    "name": "name",
}
```

Then:

```
text
user input
  ↓
validation
  ↓
allowlist
  ↓
safe SQL fragment
```

Do not weaken the security model you established on Day 8.

Day 8 specifically required an allowlist for controlled ordering because arbitrary user input should not be inserted directly into ORDER BY. fileciteturn0file7L26-L28

23. BLOCK 5 — PROJECT 1 IMPLEMENTATION

Time: 40 minutes

First inspect your current code.

Do not blindly implement a query abstraction if your Day 8 code is already clean.

Your task is:

Improve the repository's query boundary only where the actual implementation benefits from it.

Task 1 — Identify query responsibilities

Locate the current implementation of:

```
text  
GET /documents
```

Trace:

```
text  
HTTP  
↓  
Router  
↓  
Service  
↓  
Repository  
↓  
SQL  
↓  
PostgreSQL
```

Document where:

- limit is validated
- offset is validated
- filter is validated
- sort field is validated
- SQL is constructed
- SQL parameters are bound

24. Task 2 — Introduce a query representation

If appropriate for the current codebase, create something conceptually similar to:

```
text
DocumentQuery
```

containing:

```
text
document_type
limit
offset
sort_field
sort_order
```

Use the project's existing Pydantic/model conventions rather than introducing an unrelated framework.

25. Task 3 — Refactor repository query handling

Aim for:

```
text
DocumentService
  ↓
DocumentQuery
  ↓
DocumentRepository
  ↓
safe SQL
  ↓
PostgreSQL
```

The repository should be capable of interpreting the query object without multiplying repository methods.

26. Task 4 — Preserve API behavior

Existing API behavior must continue working:

```
text
GET /documents
GET /documents?limit=...
GET /documents?offset=...
GET /documents?document_type=...
GET /documents?sort=...&order=...
```

Do not add unnecessary new endpoints.

27. Task 5 — Test regression

Your Day 8 tests should continue passing.

The important test is:

Refactoring the query implementation must not change the API contract.

28. BLOCK 6 — API RELIABILITY

Time: 15 minutes

This is the new backend depth component.

Do not add Redis or distributed infrastructure.

Study and test failure behavior.

29. DATABASE UNAVAILABLE

Ask:

What happens if PostgreSQL is unavailable when GET /documents executes?

Trace:

```
text
HTTP request
↓
Router
↓
Service
↓
Repository
↓
connection failure
↓
exception
↓
API error handling
↓
HTTP response
```

Understand why the application must not return:

```
json
{
  "status": "success"
}
```

when persistence actually failed.

30. TIMEOUTS

Understand why external operations should not wait indefinitely.

For a database request:

```
text
request
↓
database operation
↓
timeout
↓
failure handling
```

You do not need sophisticated distributed timeout infrastructure today.

Understand the concept.

31. IDEMPOTENCY — CONCEPTUAL

Study the concept through your existing API.

Consider:

```
text
DELETE /documents/{id}
```

What happens if a client retries the request?

Think about:

```
text
first request
↓
delete

network failure
↓
client doesn't know whether it succeeded

retry
↓
delete again
```

This leads to the concept of idempotent operations.

You don't need to redesign DELETE today.

Just understand the interview concept.

32. RELIABILITY TESTS

Add targeted tests where appropriate.

At minimum reason through:

Test A

Database unavailable.

Expected:

```
text
request fails
```

rather than fake success.

Test B

Repository throws an unexpected exception.

Verify your existing global error behavior remains intact.

Test C

Invalid query parameters.

Verify validation occurs before unsafe database operations.

Test D

Valid query abstraction produces the same result as the previous implementation.

33. BLOCK 7 — SYSTEM DESIGN

Time: 10 minutes

Design:

Document Hierarchy Service

This connects today's tree knowledge to Project 1.

Imagine engineering artifacts have relationships:

```
text
Project
|
+-- Requirement
|
+-- Design
|
```

```
+-- Code Change
|
+-- Test
```

This naturally forms a hierarchy.

34. SYSTEM DESIGN REQUIREMENTS

Design APIs conceptually for:

```
text
Create node
Get node
List children
Get descendants
```

Think about:

```
text
Requirements
↓
Scale
↓
API
↓
Data Model
↓
Hierarchy representation
↓
Traversal
↓
Indexes
↓
Consistency
↓
Reliability
↓
Monitoring
↓
Cost
```

Do not implement this service.

This is a design exercise.

35. CONNECT IT TO PROJECT 1

Your flagship project's ultimate traceability goal is:

```

text
Requirement
  ↓
Code
  ↓
Test
  ↓
Result

```

That makes hierarchical/graph-like relationships an important future concept.

But Day 9 is only introducing the reasoning, not adding a graph database.

36. BLOCK 8 — INTERVIEW DRILL

Time: 10 minutes

Answer aloud without notes.

Trees

1. What is a tree?
2. What is a binary tree?
3. What is the difference between depth and height?
4. What is DFS?
5. What is BFS?
6. Why does DFS naturally use a stack?
7. Why does BFS naturally use a queue?
8. What are preorder, inorder and postorder?
9. What is the complexity of traversing a binary tree?
10. Why is recursive DFS $O(h)$ auxiliary space?

Backend

11. Why shouldn't every filter combination become a repository method?
12. What is a query object?
13. Where should query validation happen?
14. Where should SQL construction happen?
15. Why should SQL remain outside the router?
16. How do you safely construct dynamic ORDER BY?
17. What happens if PostgreSQL becomes unavailable?

18. Why are database timeouts important?

19. What is idempotency?

20. Why can retries create problems?

37. GOOGLE-STYLE PROJECT INTERVIEW

You should also be able to answer:

"Walk me through GET /documents from request to PostgreSQL."

Your answer should now look like:

```
text
HTTP Request
  ↓
FastAPI Router
  ↓
Pydantic validation
  ↓
DocumentQuery
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
Validated query construction
  ↓
Parameterized SQL
  ↓
PostgreSQL
  ↓
Rows
  ↓
Domain/API representation
  ↓
HTTP response
```

Then answer:

"What happens if the database fails?"

and:

"What happens if the client sends an invalid sort field?"

and:

"How would you avoid creating 20 repository methods for every filter/sort/pagination combination?"

Those are the important Day 9 engineering questions.

38. GOOGLE-INTERVIEW-PREP REPOSITORY

Add Day 9 material.

Suggested structure:

```

text
day9/
├── dsa/
│   ├── trees_notes.md
│   ├── dfs_bfs_notes.md
│   ├── binary_tree_preorder.md
│   ├── binary_tree_level_order.md
│   └── maximum_depth_binary_tree.md
├── backend/
│   ├── query_abstraction.md
│   └── api_reliability.md
├── system_design/
│   └── document_hierarchy_service.md
├── interview/
│   └── day9_interview_questions.md
└── README.md

```

Only create files for work you actually complete.

This follows the existing repository organization rather than mixing Day 9 material into earlier days.

39. PROJECT 1 DOCUMENTATION

Do not rewrite the entire README.

Day 8 already established documentation for:

- pagination
- filtering
- sorting
- validation
- query parameters
- ordering guarantees
- architecture. fileciteturn0file7L24-L25

For Day 9, add/update only:

Query abstraction

```

text
HTTP query parameters
    ↓
validated query representation
    ↓
service
    ↓
repository
    ↓
PostgreSQL

```

Reliability

Document:

- database failure behavior
- validation failures
- unexpected exceptions
- timeout considerations
- current limitations

Again:

Do not claim production-grade reliability yet.

40. GIT DISCIPLINE

Do not manufacture commits.

Potential commits only if your actual changes correspond:

```
text
feat: add tree traversal solutions
```

```
text
refactor: introduce document query abstraction
```

```
text
test: cover document query behavior
```

```
text
test: cover database failure behavior
```

```
text
docs: document query abstraction and reliability
```

The commit history should describe actual engineering changes.

41. DAY 9 DEFINITION OF DONE

DSA

- ☐ Tree fundamentals understood
 - ☐ Binary tree understood
 - ☐ DFS understood
 - ☐ BFS understood
 - ☐ Preorder understood
 - ☐ Inorder understood
 - ☐ Postorder understood
 - ☐ Recursive DFS understood
 - ☐ Iterative DFS understood
 - ☐ BFS queue mechanism understood
 - ☐ Binary Tree Preorder Traversal solved independently
 - ☐ Binary Tree Level Order Traversal solved independently
 - ☐ Maximum Depth solved independently
 - ☐ Complexity recorded
 - ☐ Edge cases recorded
 - ☐ Pattern recognition recorded
 - ☐ Solutions explained verbally
-

Backend

- ☐ Current Day 8 query implementation inspected
 - ☐ Query responsibilities identified
 - ☐ Router/service/repository boundaries preserved
 - ☐ Query abstraction understood
 - ☐ Query representation implemented if justified by actual code
 - ☐ No unnecessary repository-method explosion
 - ☐ Parameterized values preserved
 - ☐ Sort-field allowlist preserved
 - ☐ Existing pagination behavior preserved
 - ☐ Existing filtering behavior preserved
 - ☐ Existing ordering behavior preserved
-

Reliability

- ☐ PostgreSQL failure path understood
 - ☐ Unexpected exception path understood
 - ☐ Timeout concept understood
 - ☐ Idempotency concept understood
-

- ☐ Relevant failure tests added
 - ☐ Existing centralized error handling preserved
 - ☐ No fake success responses
-

System Design

- ☐ Document hierarchy designed
 - ☐ API considered
 - ☐ Data model considered
 - ☐ Traversal strategy considered
 - ☐ Scaling considerations discussed
 - ☐ Reliability discussed
 - ☐ Monitoring discussed
 - ☐ No graph database implemented prematurely
-

Documentation

- ☐ Day 9 DSA notes created
 - ☐ Backend notes created
 - ☐ Reliability notes created
 - ☐ System design notes created
 - ☐ Interview questions created
 - ☐ Project 1 documentation updated only where necessary
-

42. WHAT NOT TO DO ON DAY 9

This is particularly important for your 12-month strategy.

Do NOT start:

- Redis
 - Docker
 - Kubernetes
 - GCP
 - React
 - MLflow
 - PyTorch
 - embeddings
 - vector database
 - RAG
-

- agents
- LangChain/LangGraph
- microservices
- graph database

You are still building the SWE foundation that the later AI/ML work will depend upon.

Day 8 itself explicitly placed Redis, Docker, Kubernetes, GCP, RAG, embeddings, agents, MLflow and other infrastructure outside its scope. fileciteturn0file7L40-L40

43. THE MOST IMPORTANT DAY 9 ENGINEERING LESSON

Day 8's central lesson was:

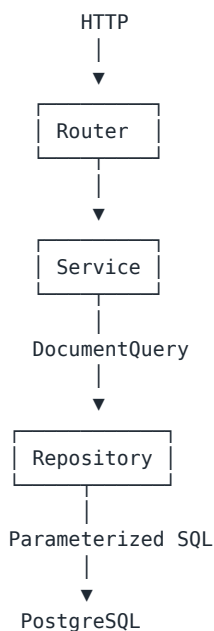
Separate HTTP concerns, business logic and persistence/query logic. fileciteturn0file7L41-L42

Day 9 builds one level deeper:

A clean repository boundary should scale in complexity without turning every combination of filters, sorting and pagination into a new method.

So your mental model becomes:

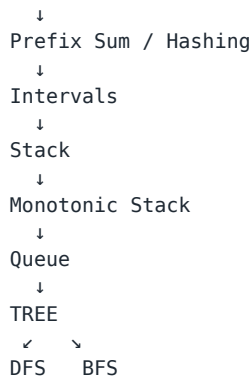
text



And on the DSA side:

```

text
Arrays
↓
Two Pointers
↓
Sliding Window
  
```



That is the correct Day 9 progression from your completed Day 8 state—not a restart of the earlier material.

Day 9 success criterion

By the end of Day 9, you should be able to simultaneously explain:

"This is a tree problem, and I know whether DFS or BFS is appropriate."

and:

"This HTTP query should be represented as structured query data, validated at the application boundary, passed through the service, safely translated into repository SQL, and executed against PostgreSQL without coupling HTTP concerns to persistence."

That combination keeps the preparation aligned with your stated target of Software Engineer first, deeply specialized in AI/ML + GenAI, while staying inside the Month-1/Month-2/early-Month-3 engineering sequence rather than jumping ahead.

Part II — Supplementary Study Aids

These sections are generated as learning support. They do not replace, alter, or paraphrase the verbatim Day 9 conversation reproduced in Part I.

Chapter 1 — Revision Summary

Day 9 establishes the next step after Days 1–8: move into trees and BFS/DFS while deepening Project 1's repository/query boundary and reliability reasoning. The work remains in the SWE/backend foundation phase and intentionally avoids premature distributed infrastructure or AI/ML platform work.

Key revision points

- Do not repeat completed DSA patterns or Day 8 query features.
- Use repository evidence before modifying Project 1.
- Trees introduce hierarchical reasoning; DFS is depth-oriented and BFS is level-oriented.
- Query abstraction prevents combinations of filters, sorting, and pagination from causing repository-method explosion.
- Reliability includes explicit failure behavior, timeouts, retries, and idempotency concepts.

Chapter 2 — DSA Revision Summary

The central recognition skill is choosing traversal strategy from the problem's structure: DFS for depth/subtree exploration and BFS for level-oriented processing. Tree traversal is also the bridge from earlier stack/queue knowledge toward general graph algorithms.

Chapter 3 — Backend Revision Summary

The backend lesson is to preserve Router Service Repository separation while representing query intent as structured data. Safe SQL construction requires parameterized values and an allowlist for SQL identifiers such as sort fields.

Chapter 4 — Completion Summary

Day 9 is complete when the new tree/DFS/BFS DSA work, query-boundary reasoning, reliability testing, system-design exercise, documentation, and Git evidence all exist without duplicating earlier work.

Glossary

Binary tree — A tree structure in which each node has at most two children.

BFS — Breadth-First Search; traversal that processes nodes level by level.

DFS — Depth-First Search; traversal that explores deeply along a branch before moving to siblings.

DocumentQuery — The structured query representation proposed for filters, pagination, and ordering.

Idempotency — The property that repeating an operation produces the same intended effect as performing it once.

Repository — The persistence/query boundary responsible for database interaction and record mapping.

Tree — A hierarchical, connected, acyclic structure used for parent/child relationships.

Index

API reliability

BFS

binary tree

DFS

DocumentQuery

idempotency

pagination

PostgreSQL

query abstraction

repository

SQL injection

stack

tree traversal

system design